

Developing with the 6500 RESTful API

This guide is an introduction to the 6500 REST API. It is intended for application developers who wish to develop applications for the 6500 NE or SIM's. The guide includes an overview of the REST API, sample code for Python and some typical REST calls. Java and Android detailed code is available in "Sample code" section.

6500 REST API

The 6500 REST API is based on the YANG API model. The YANG model has two types of resources: datastore and operations. The datastore resources (/api/v1/datastore/...) represent information about the 6500 network element that can be retrieved or modified. The operations resources (/api/v1/operations/...) are actions that can be performed on the 6500 element.

HTTP Methods

The following HTTP methods are supported:

GET	Used to retrieve datastore resources
PATCH	Used to modify datastore resources
DELETE	Used to delete an existing resource
PUT	Used to create a new resource
POST	Used to invoke operation resources

Mapping of HTTP Methods to TL1

People who are familiar with TL1 must know that mapping of HTTP methods has been directly done with the actions so that to be restful.

GET	retrieve (RTRV)
PATCH	update (ED)
DELETE	delete (DLT)
PUT	create (ENT)
POST	Rpc's that triggers an action/operation

6500 Rest API's URI Naming convention

Rest API's nomenclature/URI definitions are synchronous with YANG –

For HTTP GET/PUT/PATCH/DELETE -

/api/v1/datastore/module[/container1.../containerN]/[AID]

/api/v1/datastore/module[/container1.../containerN]/[AID]/leafParam

For HTTP POST -

/api/v1/operations/rpc/[AID]

Retrieve/GET API's e.g.–

/api/v1/datastore/shelves/shelf/shelf-1

Here, Module = shelves

Container = shelf, there can be 'N' number of containers

AID = "shelf-1"

How RestAPI's modeled in 6500

6500 itself is an entity or we can say an object which has many sub-objects like facilities, inventories, alarms, PM's etc.

All the objects are resource and for each resource there is a REST API to operate.

Thus REST API's are URI's to **the resource** itself on which the standard HTTP methods are used to trigger actions or retrieve/modify the configurations.

Login Post API Request

The 6500 Login POST Rest API is required for login into shelf which supports basic authentication. To execute a Login Post REST API call the request header must include an '**Content-Type**' header with value as "**application/x-www-form-urlencoded**". User name and password must be given in Post body as "**userid=<username>&password=<actualpassword>**". See the sample Python code below for examples.

Login command syntax for emulation cloud is:

<https://developer.ciena.com/lab/api/xxx/api/login>

For actual NE syntax is:

<https://ipaddress:8443/api/login>

In Login request reply [Response header] would contain after successful login, there would be two headers which developers need to store for subsequent requests in there code i.e. Headers are

- i) **X-XSRF-TOKEN**
- ii) **SET-COOKIE**

Request Header for subsequent request

The 6500 REST API all subsequent request **X-XSRF-TOKEN header** is used and **Cookie Header** should be created with SET-Cookie header value. See the sample Python below for examples.

The 6500 REST API also require **Content-Type** header [POST/PATCH/PUT request] in the request header value to **application/json**, this to provide Parameter content Type [i.e. to provide Body format in json] .

API also uses query arguments. See '**Query Request Parameters**' for more details.

Query Request Parameters

The 6500 REST API supports 2 query parameters for GET calls: content-type, body and AID.

Body (optional): Some commands may require content of body for post/put/patch operations

AID parameter: This defined AID of command. AID example is Shelf-<shelfid>-All, it varies on command user gives

A couple things to note with parameters:

Responses to requests are in JavaScript Object Notation (JSON) only. User **MUST** provide 2 query parameters, if not provided then REST API would fail to provide results giving 400 Request Failure response.

Following is example response with input URL.

URL: <https://host1/api/v1/datastore/equipment/inventory/SHELF-1-ALL>

Response:

```
[
  {
    "aid": "AP-1-1",
    "ctype": "Access Panel, Converged (SONET)",
    "pec": "NTK505MB",
    "rel": "",
    "clei": "SIMAP",
    "ser": "",
    "mdat": "UNKNOWN"
  },
  {
    "aid": "2XOSC-1-1",
    "ctype": "OSC w/WSC 2 Port SFP 2 Port 10/100 BT",
    "pec": "NTK554BA",
    "rel": "00 ",
    "clei": "CLEI_8161",
    "ser": "NNTMRT072GW3",
    "mdat": "0-01",
    "cardpwr": "FULL",
    "age": "00-000-18-11",
    "on-since": "00-000-00-03"
  },
  {
    "aid": "EMPTY-1-1-1"
  },
  {
    "aid": "EMPTY-1-1-2"
  },
  {
    "aid": "LIM-1-2",
    "ctype": "MLA C-Band",
    "pec": "NTK552BA",
    "rel": "02A",
```

```

    "clei": "CLEI_8162",
    "ser": "NNTMLIM00001",
    "mdat": "0-51",
    "cardpwr": "FULL",
    "age": "00-000-18-10",
    "on-since": "00-000-00-03"
  },
  .... Remaining inventory ports
]

```

REST API Syntax

REST API calls require the following:

Request URL in the format:

For GET, PATCH, PUT and DELETE requests:

`https://<ip_address>:8443/api/v1/datastore/<rest_resource>`

or

For POST requests:

`https://<ip_address>:8443/api/v1/operations/<rest_resource>`

where

<ip_address> is the 6500 IP address

8443 is the secure HTTP port number. This value is must required for actual NE.

<rest_resource> is the requested resource

Request Header - which includes an X-XSRF-TOKEN , COOKIE and Content-Type [optional]

Header.

Request Body – JSON data for PATCH, POST and PUT calls

URL format when accessing simulated 6500 devices through the CEC portal:

The CEC portal does not provide an IP address for accessing simulated 6500 devices. Instead a device path value is provided. The device path format is ***developer.ciena.com/lab/api/xx***. Where xx is a unique numeric value for that SIM instance

When using a devices path the Request URL uses the format:

https://developer.ciena.com/lab/api/xx/api/v1/datastore/<rest_resource>

or

https://developer.ciena.com/lab/api/xx/api/v1/operations/<rest_resource>

SSL Certificates

The 6500 network element and the CEC portal provide self-signed SSL certificates.

6500 WebSocket Interface

The 6500 API includes a 6500 interface to monitor autonomous messages. To access the WebSocket interface from a web browser use one of the following URLs:
After logging into shelf through Swagger you can connect to 6500 WebSocket, it will take to URL

<https://<IP address>/eventlistener.html> (Real 6500 NE)

or Emulation cloud after logging swagger

<https://developer.ciena.com/lab/6500/<TokenID>/SiteName> (Simulated 6500 NE)

To access the WebSocket interface from a client application follows steps and use one of

following URLs:

- 1) First login into shelf by using API <https://ipaddress:8443/api/login>

Get Token header field & also cookie header field to attach.

- 2) <wss://<IP address>:8443/status?-xsrf=<xsrfTokenValue>> (Real 6500 NE)

OR emulation cloud

- 1) First login into shelf by using API <https://developer.ciena.com/lab/api/xxx/api/login>

Get Token header field & also cookie header field to attach.

- 2) <wss://developer.ciena.com/lab-notification/ome/xxx/status?-xsrf=<xsrfTokenValue>>
(simulated 6500NE)

Python Sample Code

Python Version 2.7

With Python 2.7 code, the [requests](#) library is recommended for all REST operations & all warnings have been disabled for sample code testing.

Sending a simple LOGIN request

Below is a simple function to send an HTTPS post Login request to a 6500 NE.

```
import requests
from urlparse import urlunparse
from requests.auth import HTTPBasicAuth

def login_post_request(host, username, password, resource):

    # build the URL
    url = urlunparse(('https', host, resource, None, None, None))
    data = {'userid':username,'password':password }
    print "POST %s" % url
    # This header is needed by 6500 Login request.
    headers ={'Content-Type' : 'application/x-www-form-urlencoded'}

    urllib3.disable_warnings(InsecureRequestWarning)
    urllib3.disable_warnings(InsecurePlatformWarning)
    urllib3.disable_warnings(SNIWarning)
    resp = requests.post(url,data=headers = headers ,verify = False)
    return resp
```

Sending a simple GET request

Below is a simple function to send an HTTPS GET request to a 6500 NE.

```
def simple_get_request(host, resource, cookieValue,XSRFtokenValue):
    """
        A simple Get Request. GET the value from the URL.
        :param host: The URL to the server
        :param resource: The URI representing the API operation to perform
        :param cookieValue CookieValue which we got from Login Request
        :param XSRFtokenValue X-XSRF-TOKEN header value which we got from Login
        Request
        :return: A Requests Response type object
    """
    # build the URL
    url = urlunparse(('https', host, resource, None, None, None))
    print "GET: %s" % url

    headers = { 'X-XSRF-TOKEN' : XSRFtokenValue , 'Cookie' : cookieValue}

    urllib3.disable_warnings(InsecureRequestWarning)
    urllib3.disable_warnings(InsecurePlatformWarning)
    urllib3.disable_warnings(SNIWarning)
    resp = requests.get(url, headers = headers, verify=False)
    return resp
```

This function requires a valid hostname or IP address, resource URI, cookieValue [X-XSRF-TOKEN header value] & [SET-COOKIE header value]. The function encodes COOKIE & X-XSRF-HEADER in the request header.

Running the function will return a Requests Response type object which can print json by calling the "json()" function:

```
host = 'developer.ciena.com/lab/api/xx' # xx is the unique instance
ID for the SIM
resp =
simple_get_request(host, '/api/v1/datastore/adjacencies/adjacency/SH
ELF-1-ALL', <cookieValue>, <xsrHeaderValue>)
print resp.json()
```

URL: <https://developer.ciena.com/lab/api/xx/api/v1/datastore/ws-system/host-name>

```
[
  {
    "aid": "ADJ-1-1-1",
    "adj-type": "LIM",
    "prov-fe-addr": "xcr-1-2-3",
    "prov-addr-form": "TID-SH-SL-PRT",
    "disc-fe-addr": "",
    "disc-addr-form": "NULL",
    "adj-stat": "DERIVED",
    "layer": "PHYS",
    "clfi": "",
    "pst": "IS"
  },
  {
    "aid": "ADJ-1-1-2",
    "adj-type": "UNKNOWN",
    "prov-fe-addr": "",
    "prov-addr-form": "TID-SH-SL-PRT",
    "disc-fe-addr": "",
    "disc-addr-form": "NULL",
    "adj-stat": "UNVERIFIED",
    "layer": "PHYS",
    "clfi": "",
    "pst": "IS"
  },
  .... Remaining adjacencies
]
```

Sending a PUT request

Below is a function to send an HTTPS PUT request to the 6500 NE.

```
def simple_put_request(host, resource, jsonData, cookieValue, XSRFtokenValue):
    """
    A simple Put Request. PUT value to the URL.
    :param host: The URL to the server
    :param resource: The URI representing the API operation to
    perform
    :param jsonData: Data to to PUT (Create Object)
    :param cookieValue CookieValue which we got from Login Request
    :param XSRFtokenValue X-XSRF-TOKEN header value which we got
    from Login Request
    :return: A Requests Response type object
    """
    # build the URL
    url = urlunparse(('https', host, resource, None, None, None))
    print "PUT: %s" % url

    headers = { 'X-XSRF-TOKEN' : XSRFtokenValue , 'Cookie' :
    cookieValue}

    urllib3.disable_warnings(InsecureRequestWarning)
    urllib3.disable_warnings(InsecurePlatformWarning)
    urllib3.disable_warnings(SNIWarning)
    resp = requests.put(url, json = jsonData , headers = headers,
    verify=False)
    return resp
```

This function requires a valid hostname or IP address, resource URI, change data [jsonData], cookieValue [X-XSRF-TOKEN header value] & [SET-COOKIE header value]. The function encodes COOKIE & X-XSRF-HEADER in the request header

When run it will print the URL and JSON response.

The following exam add to add SPLI adjacency by PUT request with data

```
host = 'developer.ciena.com/lab/api/xx' # xx is the unique instance ID for
the SIM
resource = '/api/v1/datastore/adjacencies/spli/SHELF-1'
data = { "fend-ip-addr": "192.168.0.20", "spli-id": "Test-2" }
resp = sample_put_request(host, resource, data, cookieValue, xsrfToken)
print resp.json()
```

Response:

```
{"NOERROR": "API executed successfully"}
```

Use the GET command to see the data change(s) & and to see the spli adjacencies changes request:

```
host = 'developer.ciena.com/lab/api/xx' # xx is the unique instance ID for
the SIM
resp = simple_get_request(host, '/api/v1/datastore/adjacencies/spli/SHELF-1',
<cookieValue>, <xsrfHeaderValue>)
```

URL: <https://developer.ciena.com/lab/api/xx/api/v1/datastore/adjacencies/spli/SHELF-1>

Response:

```
[
  {
    "shelf": "SHELF-1",
    "index": "1",
    "spli-id": "yyuui-1",
    "keyformat": "TID",
    "splistatus": "RELIABLE",
    "numsplimatches": "2",
    "fend-ip-addr": "48.0.0.3",
    "commstype": "LOCAL",
    "commsstate": "IN_USE",
    "commsuserovrd": "FALSE"
  },
  {
    "shelf": "SHELF-1",
    "index": "3",
    "spli-id": "TEST-2",
    "keyformat": "NODENAME",
    "splistatus": "RELIABLE",
    "numsplimatches": "0",
    "fend-ip-addr": "192.168.0.20",
    "commstype": "TCPSSH",
    "commsstate": "REQUESTED",
    "commsuserovrd": "FALSE"
  }
]
```

Sending a POST request

Below is a function to send an HTTPS POST request to the 6500 NE

```
def simple_post_requestParam(host, resource, jsonData,cookieValue,XSRFtokenValue):

    A Simple POST request to the API Server
    :param host: The URL to the server
    :param resource: The URI representing the API operation to perform
    :param data: A JSON object holding the data for the POST request.
    : param cookieValue = Cookie received from Post login response header
    : param XSRFtokenValue = X-XSRF-Token header response from post login request
    :return: A Requests Response type object

    # build the URL
    url = urlunparse(('https', host, resource, None, None, None))
    print "POST %s" % url
    headers = { 'X-XSRF-TOKEN' : XSRFtokenValue , 'Cookie' : cookieValue}
    urllib3.disable_warnings(InsecureRequestWarning)
    urllib3.disable_warnings(InsecurePlatformWarning)
    urllib3.disable_warnings(SNIMissingWarning)

    resp = requests.post(url,data=json.dumps(jsonData),headers = headers ,verify =
    False)
    return resp
```

This function requires a valid hostname or IP address, resource URI, change data, CookieValue & XSRFToken received from Post response header.

When run it will print the URL and JSON response. Use the GET command to see the data change(s).
The following example add configuration file list:

```
host = 'developer.ciena.com/lab/api/xx' # xx is the unique instance ID for
the SIM resource = '/api/v1/operations/adjacencies-delete-spli'
data = { 'index': '3', 'shelf': 'SHELF-1' }
resp = simple_post_requestParam(host, resource, jsonData,
cookieValue,XSRFtokenValue)
print resp.json()
```

URL: <https://developer.ciena.com/lab/api/xxx/api/v1/operations/adjacencies-delete-spli>

Response:

```
{"NOERROR": "API executed successfully"}
```

And to see the SPLI Adjacency has been deleted, Get SPLI Adjacency:

```
resource = ' /api/v1/datastore/adjacencies/spli/SHELF-1'
resp = simple_GET_example(hostName, resource,<TokenValue>,<CookieManager>)
print resp.json()
```

URL: <https://developer.ciena.com/lab/api/xxx/api/v1/datastore/adjacencies/spli/SHELF-1>

Response:

```
[
  {
    "shelf": "SHELF-1",
    "index": "1",
    "spli-id": "yyuui-1",
    "keyformat": "TID",
    "splistatus": "RELIABLE",
    "numsplimatches": "2",
    "fend-ip-addr": "48.0.0.3",
    "commstype": "LOCAL",
    "commsstate": "IN_USE",
    "commsuserovrd": "FALSE"
  }
]
```

Sending a PATCH request

Below is a function to send an HTTPS PATCH request to the 6500 NE.

```
def simple_patch_request(host, resource, jsonData, cookieValue,XSRFtokenValue):
    """
        A simple Patch Request.
    :param host: The URL to the server
        :param resource: The URI representing the API operation to perform
        :jsonData : change data which user need to patch
        : param cookieValue = Cookie received from Post login response header
        : param XSRFtokenValue = X-XSRF-Token header response from post login request
    :return: A Requests Response type object
    """

    # build the URL
    url = urlunparse(('https', host, resource, None, None, None))
    print "PATCH: %s" % url
    headers = { 'X-XSRF-TOKEN' : XSRFtokenValue , 'Cookie' : cookieValue}
    urllib3.disable_warnings(InsecureRequestWarning)
    urllib3.disable_warnings(InsecurePlatformWarning)
    urllib3.disable_warnings(SNIMissingWarning)
    resp = requests.patch(url, json = jsonData , headers = headers,
    verify=False)
    print resp.request.headers
    return resp
```

This function requires a valid hostname or IP address, resource URI, change data,cookieValue and

XSRFtokenValue .

When run it will print the URL and JSON response. Use the GET command to see the data change(s).

The following example changes the date & time of NE:

```
host = 'developer.ciena.com/lab/api/xx' # xx is the unique instance ID for
the SIM resource = '/api/v1/datastore/system/date'
data = { "current-date": "16-08-18", "current-time": "23-41-59" }
resp = simple_patch_request(host, resource, jsonData, cookieValue,
XSRFtokenValue)
print resp.json()
```

URL: <https://developer.ciena.com/lab/api/xx/api/v1/datastore/system/date>

Response:

```
{"NOERROR": "API executed successfully"}
```

And to see the date & time change give run following:

```
resource = '/api/v1/datastore/system/date'
resp = simple_get_request(host,resource,cookieValue,XSRFToken) print
resp.json()
```

URL: <https://developer.ciena.com/lab/api/xx/api/v1/datastore/system/date>

Response:

```
[
  {
    "current-date": "16-08-18",
    "current-time": "23-41-51"
  }
]
```

Sending a DELETE request

```
def simple_delete_request(host, resource, cookieValue,XSRFtokenValue):
    """
    A simple delete Request. Delete the object with AID
    :param host: The URL to the server
    :param resource: The URI representing the API operation to perform
    : param cookieValue = Cookie received from Post login response header
    : param XSRFtokenValue = X-XSRF-Token header response from post login request
    :return: A Requests Response type object
    """
    # build the URL
    url = urlunparse(('https', host, resource, None, None, None))
    print "DELETE: %s" % url
    headers = { 'X-XSRF-TOKEN' : XSRFtokenValue , 'Cookie' : cookieValue}

    urllib3.disable_warnings(InsecureRequestWarning)
    urllib3.disable_warnings(InsecurePlatformWarning)
    urllib3.disable_warnings(SNIMissingWarning)

    resp = requests.delete(url, headers = headers, verify=False)
    print resp.request.headers
    return resp         return resp
```

Below is a function to send an HTTPS DELETE request to the 6500 NE.

This function requires a valid hostname or IP address, resource URI, change data,cookieValue and XSRFtokenValue .

When run it will print the URL and JSON response. Use the GET command to see the data change(s).

The following example delete Adjacency of given AID [Note:- Adjacency should be in OOS-MA State]:

```
host = 'developer.ciena.com/lab/api/xx' # xx is the
unique instance ID for the SIM resource =
'/api/v1/datastore/adjacencies/adjacency/ADJ-1-84-23'
resp = simple_delete_request(host, resource,
cookieValue, XSRFtokenValue)
print resp.json()
```

URL: <https://developer.ciena.com/lab/api/xx/api/v1/datastore/adjacencies/adjacency/ADJ-1-84-23>

Response:

```
{"NOERROR": "API executed successfully"}
```

And to see the adjacency has been deleted Get all Adjacencies by following API:

```
resource =
'/api/v1/datastore/adjacencies/adjacency/SHELF-1-ALL'
resp =
simple_get_request(host,resource,cookieValue,XSRFToken)
```

URL: <https://developer.ciena.com/lab/api/xx/api/v1/datastore/adjacencies/adjacency/SHELF-1-ALL>

Response:

```
[
  {
    "aid": "ADJ-1-10-16",
    "adj-type": "WSS",
    "prov-fe-addr": "",
    "prov-addr-form": "TID-SH-SL-PRT",
    "disc-fe-addr": "",
    "disc-addr-form": "NULL",
    "adj-stat": "UNVERIFIED",
    "layer": "PHYS",
    "clfi": "",
    "port-label": "Switch 7 Out",
    "pst": "IS"
  }
  {
    "aid": "ADJ-1-10-22",
    "adj-type": "LIM",
    "prov-fe-addr": "xcr-1-13-6",
    "prov-addr-form": "TID-SH-SL-PRT",
    "disc-fe-addr": "",
    "disc-addr-form": "NULL",
    "adj-stat": "DERIVED",
    "layer": "PHYS",
    "clfi": "",
    "port-label": "Common Out",
    "pst": "IS"
  },
  ... remaining adjacencies
]
```

Sending a simple Logout POST request

```
def simple_post_request(host, resource, cookieValue,XSRFtokenValue):  
  
    A Simple POST request to the API Server  
    :param host: The URL to the server  
    :param resource: The URI representing the API operation to perform  
    :param data: A JSON object holding the data for the POST request.  
    : param cookieValue = Cookie received from Post login response header  
    : param XSRFtokenValue = X-XSRF-Token header response from post login  
request  
    :return: A Requests Response type object  
  
    # build the URL  
    url = urlunparse(('https', host, resource, None, None, None))  
    print "POST %s" % url  
    headers = { 'X-XSRF-TOKEN' : XSRFtokenValue , 'Cookie' : cookieValue}  
    urllib3.disable_warnings(InsecureRequestWarning)  
    urllib3.disable_warnings(InsecurePlatformWarning)  
    urllib3.disable_warnings(SNIMissingWarning)  
  
    resp = requests.post(url, headers = headers ,verify = False)  
    return resp
```

Below is a function to send an HTTPS Simple POST request to the 6500 NE.

This function requires a valid hostname or IP address, resource URI, CookieValue & XSRFToken received from Post response header.

When run it will print the URL and JSON response. Try login into shelve by any of commands above user will get 401 not authorized.

The following example add configuration file list:

```
host = 'developer.ciena.com/lab/api/xx' # xx is the unique instance ID for  
the SIM resource = '/api/logoff'  
resp = simple_post_request(host, resource, cookieValue,XSRFtokenValue)  
print resp.json()
```

URL: <https://developer.ciena.com/lab/api/xxx/api/logoff>

Response:

200OK

Connect to WebSocket Notification Interface

Use the sample code below to establish a secure websocket connection and print the received messages. The websocket is closed when the user enters a Ctrl C.

```
import base64
import json
from ws4py.client.threadedclient import WebSocketClient

def simple_websocket_example(host, cookieVal, tokenVal):
    headers = [{"cookie", cookieVal}]
    protocols = ['6500-ao-json']
    ws = MyClient('wss://' + host + '/status?-xsrftoken=' + tokenVal , protocols=protocols,
headers=headers)
    try:
        # Print input URL
        print('URL: ' + str(ws.url) + '\n')
        ws.connect()
        ws.run_forever()
    except KeyboardInterrupt:
        ws.close()

class MyClient(WebSocketClient):
    def received_message(self, m):
        data = m
        if m is not None:
            d = str(m)
            js = json.loads(d)
            # Print json message in readable format
```

First user has to login to 6500 NE to get X-XSRF-TOKEN Header value & cookie Value to connect to websockets.

This above function requires a valid portal URL or IP address , cookieValue & XSRFTOKEN.

```
simple_websocket_example('developer.ciena.com/lab-notification/ome/xxx', cookieValue,
XSRFTOKEN)
```

Note that the portal URL uses developer.ciena.com/lab-notification/ome/<virtual device ID>

Sample Output of URL

URL: wss://developer.ciena.com/lab-notification/ome/172/status?-xsrftoken=<xsrftokenValue>

Sample Output also depend upon action done on Simulator or <virtual device ID>